

week 3: ROC curves, Naive Bayes and Entropy

<http://mlvu.github.io>

February 18, 2025

1 Exam questions

At this point in the course, it's a good idea to have a look at the practice exam. The homework prepares you for the exam, but the format of the homework questions is different from that of the exam questions. Here are some questions you should be able to answer based on the material covered so far.

Canvas offers a complete practice exam, a documentation of how the exam works and two Canvas quizzes which randomly generate exams.

1. What is the advantage of gradient descent over random search?
 - A In gradient descent, parallel searches are allowed to communicate.
 - B Gradient descent is less likely to get stuck in local minima.
 - C Gradient descent computes the direction of steepest descent, random search approximates it. ✓
 - D Gradient descent is easier to parallelise.
2. Why is accuracy a bad loss function to use in gradient descent?
 - A It is expensive to compute.
 - B It makes the gradient zero almost everywhere. ✓
 - C It is unreliable in situations with high class imbalance.
 - D The confidence interval is high on small test sets.
3. I'm performing spam classification. I represent each email by three numbers: how often the word *pill* occurs, how often the word *hello* occurs and how often the word *congratulations* occurs. What are these three attributes called?
 - A The instances
 - B The classes
 - C The features ✓
 - D The principal components

Here we see the derivation of the gradient of the squared-error loss for linear regression.

$$\frac{\partial \frac{1}{2} \sum_i (f(x_i) - y_i)^2}{\partial b} = \frac{1}{2} \frac{\partial \sum_i (x_i w + b - y_i)^2}{\partial b} \quad (1)$$

$$= \frac{1}{2} \sum_i \frac{\partial (x_i w + b - y_i)^2}{\partial b} \quad (2)$$

$$= \frac{1}{2} \sum_i \frac{\partial (x_i w + b - y_i)^2}{\partial (x_i w + b - y_i)} \frac{\partial (x_i w + b - y_i)}{\partial b} \quad (3)$$

$$= \sum_i (x_i w + b - y_i) \frac{\partial (x_i w + b - y_i)}{\partial b} \quad (4)$$

$$= \sum_i (x_i w + b - y_i) \quad (5)$$

4. To get from line (1) to line (2), we use the
A Chain rule **B** Product rule **C** Exponent rule **D** Sum rule ✓
5. To get from line (2) to line (3), we use the
A Chain rule ✓ **B** Product rule **C** Exponent rule **D** Sum rule
6. In line 5, the correct result is
A $\sum_i (x_i^2 w + b - y_i)$
B $\sum_i x_i (x_i w + b - y_i)$
C $\sum_i (x_i w + b - y_i)$ ✓
D $\sum_i (x_i w + b - y_i)^2$

2 ROC Curves and confusion matrices

In this section, we'll look at evaluation metrics. This section covers the lecture *Evaluation* and section 2.2 of Peter Flach's *Machine Learning* (see the PDF on Canvas).

As a running example, we will use the following classification dataset:

	x_1	x_2	label
a	1	0	Neg
b	2	0	Pos
c	3	0	Pos
d	$1/2$	1	Pos
e	2	2	Pos
f	0	3	Neg
g	1	3	Neg
h	2	3	Neg

If you have trouble with the exercises below, we recommend drawing the data in its feature space, and drawing the decision boundary of each classifier.

2.1 A linear classifier

question 1: As a first classifier, c_{lin} , we will use a diagonal line crossing all the points where $x_1 = x_2$. Points above this line will be classified as Neg, points below or on the line as Pos. How do we describe this classifier mathematically? Fill in the blanks:

$$c_{\text{lin}}(x_1, x_2) = \begin{cases} \text{Pos} & \text{if } x_1 \geq x_2 \\ \text{Neg} & \text{otherwise} \end{cases}.$$

The diagonal that the question asks for (the decision boundary) is the set of points where $x_1 = x_2$. If $x_1 > x_2$, we're below this line, so we should classify the point as Pos.

c_{lin} is a simple classifier. Let's try a more complex one before we move on.

question 2: Let c_2 be the classifier whose decision boundary crosses the points (0, 1) and (1, 1.5), with the class Neg above the line. Draw this line first. How do we define this classifier?

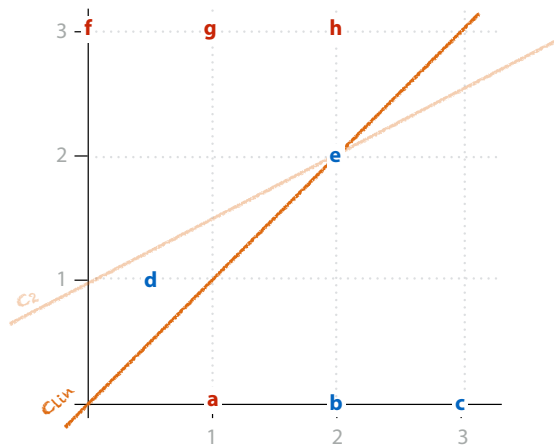
$$c_2(x_1, x_2) = \begin{cases} \text{Pos} & \text{if } -0.5x_1 + x_2 - 1 \leq 0 \\ \text{Neg} & \text{otherwise} \end{cases}$$

We could fill the given values in to the equation $ax_1 + bx_2 + c = 0$ and solve for zero. But that would give us two problems: it's a lot of work, and we have two equations for three variables. There are infinitely many pairs a and b that would give us this decision boundary. Let's make things easier

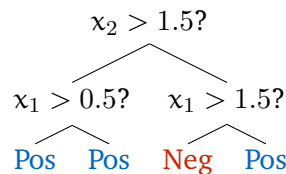
by assuming that b is 1, and rewriting to $x_2 = -ax_1 - c$. This is just the familiar function for a line in the plane: $-a$ is the slope, and $-c$ is the bias.

The given points tell us that the line crosses the x_2 axis at 1, and the slope should be 0.5. Thus, $-a = 0.5$ and $-c = 1$. A quick check tells us that for a point below the line, $(0, 0)$, the quantity $-0.5x_1 + x_2 - 1$ is negative, so such points should be classified as **Pos**.

Here is a diagram of the data and the boundaries for the two classifiers:



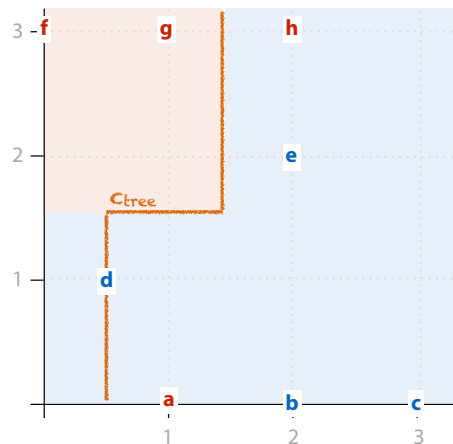
question 3: We will compare the linear classifier c_{lin} with a *decision tree* classifier, c_{tree} . Here is the decision tree:



If the inequality on the node is true, move right, otherwise move left.

This tree divides the feature space into four **segments** (A, B, C and D). To each segment we assign the majority class in that segment (using **Pos** for a draw). Label the leaves A, B, C and D with classes.

Here is a diagram of the data and the boundaries for the tree classifier:



question 4: In this exercise we don't do any training (we just evaluate the given classifiers), so we'll evaluate all metrics on a single given dataset.

If you do train, you would *split* your dataset. You would compute your evaluation metrics on the **validation data** during hyperparameter selection and on the **test data** during final testing.

Evaluating on the training data is the worst sin you can commit in Machine Learning. Can you think of a situation when you would still want to *compute* the training loss?

Getting good training loss, but poor validation loss is a sure sign of overfitting. If overfitting is likely (as it is in decision trees and neural networks), you may want to compare the training loss to the validation loss to see if overfitting explains the poor performance.

question 5: Give the *confusion matrices* for classifiers c_{lin} and c_{tree} .

c_{lin}		predicted	
		Pos	Neg
true	Pos	3	1
	Neg	1	3

c_{tree}		predicted	
		Pos	Neg
true	Pos	4	0
	Neg	2	2

question 6: From the confusion matrices, we can compute several metrics. Compute the accuracy, precision, recall, true positive rate and false positive rate

We compute these metrics from the four values of the confusion matrix: the number of true positives (TP), true negatives (TN), false positives (FP) and false negatives(FN):

		predicted	
		Pos	Neg
true	Pos	TP	FN
	Neg	FP	TN

		c_{lin}	c_{tree}
accuracy	$\frac{TP+TN}{total}$	3/4	3/4
precision	$\frac{TP}{TP+FP}$	3/4	2/3
recall	$\frac{TP}{TP+FN}$	3/4	1
true positive rate	$\frac{TP}{TP+FN}$	3/4	1
false positive rate	$\frac{FP}{TN+FP}$	1/4	1/2

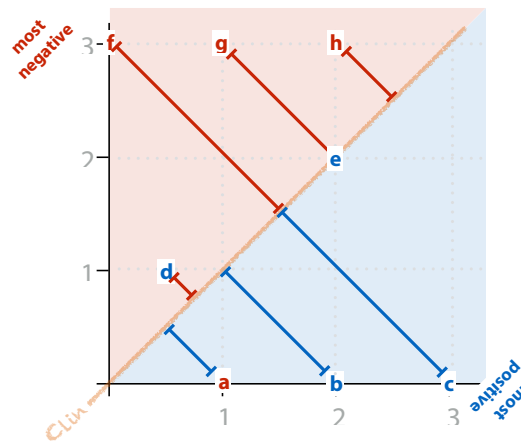
Most of these are pretty intuitive, if you work out the meaning of the various sums. For instance, the false positive rate is the proportion of actual negatives, that were predicted to be positive.

To plot a ROC and precision/recall curves, and to compute AUC metrics, we need to turn our simple binary classifier into a *ranking* classifier. That is, we don't just want classifications, we want it to rank the data from most Neg to most Pos. How we do this depends on the classifier.

question 7: How do we turn a linear classifier into a ranking classifier? Give the ranking that c_{lin} assigns to the training data.

We compute the distance to the classification boundary (the negative distance on one side, and the positive on the other), and rank points by this distance. For this course (the homework and the exam), you don't actually need to compute the distance, it's fine to draw the dataset and "measure" the distances. We'll make sure that all examples allow this approach.

Here's the sort of diagram you should end up with:



ranking:



So, for c_{lin} the ranking from negative to positive is

f g h d e a b c .

question 8: How do we turn a decision tree classifier into a ranking classifier? Give the ranking that c_{tree} assigns to the training data.

We group the points by the way the decision tree segments the feature space. We then assign each segment a class probability based on the relative frequencies of *training* data points. We sort the groups by class probability. Points within the same group are ranked equal.

For c_{tree} the ranking from negative to positive is

(f g) (h e) (a b c) (d).

Brackets indicate equal rank.

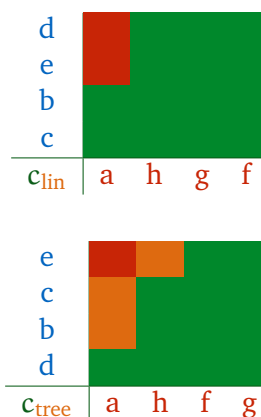
We can't tell whether our ranking is perfect, because the training data doesn't give us a correct *ranking*, only a correct *labeling*. But for some pairs of instances we *do* know how they should be ranked: all pairs of positive and negative examples.

We can visualize this in a matrix with **negative** instances on the horizontal axis, and **positive** instances on the vertical, both sorted with the most positive (according to the classifier's ranking) in the bottom left corner. The

matrix is usually colored **green** for pairs that are ranked correctly, **red** for pairs ranked incorrectly, and **yellow** for pairs that are ranked equal.

question 9:

1. Draw the coverage matrices of c_{lin} and c_{tree} .

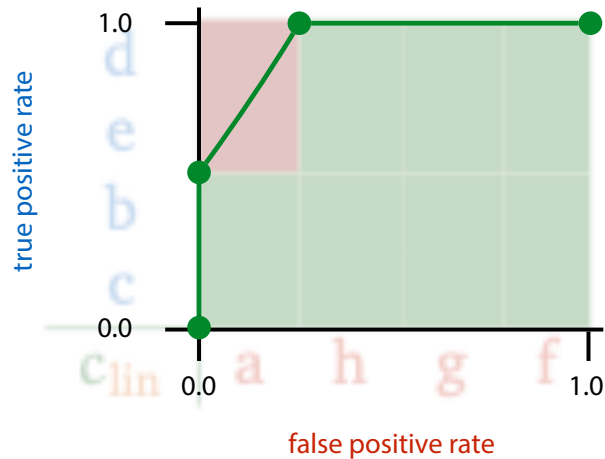


2. How many ranking errors does each classifier make on the training data? c_{lin} makes 2 ranking errors. c_{tree} makes 2.5, counting each tie as half a ranking error.
3. If we have class imbalance (one class has more instances than the other), how can we tell by the coverage matrix?

It will be non-square. The more stretched out, the higher the class imbalance.

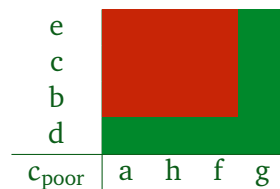
question 10: How do we convert a coverage matrix to an **ROC plot**? How does the green area in a coverage matrix relate to the ROC plot? How do they differ?

We can convert the coverage matrix to ROC space by normalizing the axes. That is, if we have a green cell at (2, 3), counting from the bottom left, we can draw a point in ROC space at $(\frac{2}{4}, \frac{3}{4})$. That is, we know we have a classifier that can achieve a true positive rate of $\frac{3}{4}$ and a false negative rate of $\frac{2}{4}$.



The green area gives us an estimate of the probability of a ranking error. The same is true of the *Area Under the Curve*. For the latter, we draw the convex hull of all achievable classifiers, and compute the area.

An important difference is that the green area in a coverage matrix may be *concave*. For instance:



The corresponding ROC curve is convex (it covers the bottom right half of the ROC space).

Since a classifier with AUC below 0.5 is pretty useless, we don't usually worry about this. The differences between the (normalized) coverage matrix and the ROC space are not important for reasonable classifiers and large datasets.

3 preliminaries: probability

If the exercises below give you any trouble we recommend that you follow the links provided to brush up a little.

question 11:

1. In a few sentences, explain the difference between the Frequentist and the Bayesian interpretation of probability. A frequentist considers a probability an objective value: a property of the universe that can be measured by repeated experimentation, like measuring the probability that a bent coin lands heads, by flipping it repeatedly. A Bayesian considers probability an expression of uncertain belief. A Bayesian can talk about the probability that “John is having an affair”. To a frequentist this is nonsense, because John is either having an affair or he isn’t.
2. What is the difference between a sample space and an event space? A sample space is a space of individual things that can happen (like a die landing on a 6) and an event space is a space of sets of things from the sample space (like a die landing on an even number). Technically, an event space should be a sigma-algebra of a sample space, but for discrete probability distributions, we can think of the event space as the powerset of the sample space. For continuous probability distributions, we can ignore the technical details.
3. Jack and Jill have a dataset of biometric data of Dutch men, and they fit a normal distribution to their height. They get a mean of 183.8 cm. Jack suggest that this means that the probability of seeing a Dutch man this tall is higher than any other height. Jill disagrees: she suggests that the more precisely we state the mean (with more digits after the decimal point) the smaller the probability will get. For the exact mean, the probability of seeing someone with that exact height is zero. Jack points to the shape of the normal distribution: it peaks at the mean, surely that means that that is where the most probable outcome is. Who is right? The key point is that the sample space is *continuous*. This means that we define the probability distribution by a probability *density* function. It assigns every single outcome (a height) a probability density, but not a probability. We get probabilities by integrating the probability density function over a range of values. For instance, the probability of seeing a person between 165 and 170 cm tall gets a probability.

In some sense John was also correct, in that we usually talk about values with a limited precision. For instance, when we say somebody’s height is 183.8 cm, we don’t mean it’s exactly 183.80000000...cm. We mean something like “it’s somewhere between 183.75 cm and

183.85 cm.” This is a range of values, so it gets a nonzero probability. In the distribution that Jack and Jill used, among all ranges of 0.1 cm wide, this is the one that gets the highest probability.

See also [this slide in the preliminaries](#).

question 12: In the following, p is a probability function and A and B are random variables. Which of the following are true?

1. Joint probability is symmetric: $p(A, B) = p(B, A)$. **True**
2. Conditional probability is symmetric $p(A | B) = p(B | A)$. **False.** This is not true in general (although it may be true for specific A and B).
3. Two random variables X and Y are conditionally independent on a third Z . Once we know X and Y , we also know the value of Z . **False.**
4. Two random variables X and Y are conditionally independent on a third Z . Once we know Z , also knowing X will tell us nothing extra about Y . **True:** conditional independence means that given the value of the conditional, X and Y are independent (so knowing the value of one reveals nothing about the other).

question 13: Assume that the probability that a given patient has diabetes is 0.01. We have a test for diabetes with a false positive rate of 0.05: if a patient has no diabetes, the test diagnoses it 5% of the time. The false negative rate is 0.1. You are a doctor, and you administer the test to a patient (knowing nothing else). The test says she has diabetes. What is the probability that she doesn't? Hint: this is a question about Bayes' rule. Reflect on the result. Is this what you would've expected? If not, where does the unexpected result come from?

Let's write down the given probabilities first. We'll use D and $\neg D$ for the events that the patient has and doesn't have diabetes respectively. We'll use T for the event of the test *saying* that the patient has diabetes and $\neg T$ for the test saying that she doesn't. We are given $p(D) = 0.01$, $p(T | \neg D) = 0.05$ and $p(\neg T | D) = 0.1$. We can derive $p(\neg T | \neg D) = 0.95$ and $p(T | D) = 0.9$. Now, we want to know $p(\neg D | T)$. We'll use Bayes' rule to reverse the conditional

probability:

$$\begin{aligned} p(\neg D | T) &= \frac{p(T | \neg D)p(\neg D)}{p(T)} \\ &= \frac{p(T | \neg D)p(\neg D)}{p(T, D) + p(T, \neg D)} \\ &= \frac{p(T | \neg D)p(\neg D)}{p(T | D)p(D) + p(T | \neg D)p(\neg D)} \\ &= \frac{0.05 \times 0.99}{0.9 \times 0.01 + 0.05 \times 0.99} \\ &= \frac{5 \times 99}{90 + 5 \times 99} \approx 0.85 \end{aligned}$$

Whether you expected this result is up to you, of course, but it's certainly not what you want from a test like this. And yet, the false positive and false negative rates don't seem extremely high. In the last line, I've multiplied everything by 100 so you can see where the imbalance comes from: **the high prior probability that someone doesn't have diabetes** dominates (in other words, we have high class imbalance). In order for the test to be reliable, the **false positive rate** needs to be low enough to make the numerator small in proportion to the denominator.¹

More practice? Follow these links:

1. <https://seeing-theory.brown.edu/> (especially this one)
2. <https://betterexplained.com/articles/a-brief-introduction-to-probability-statistics/>
3. <https://www.khanacademy.org/math/probability/probability-geometry/probability-basics/a/probability-the-basics>
4. <http://dept.stat.lsa.umich.edu/~moulib/probrefresh.pdf>

4 Naive Bayes

The following dataset represents a spam classification problem: we observe 8 emails and measure two binary features with the values **True** and False.

¹There's a subtle difference between the phrase *false positive rate* as we use it here and as we've used it previously. What we described previously (like in lecture 3) is an estimate of the fpr, computed from the test set. What we discuss here is the "actual" fpr: an unknown quantity that we would normally only be able to estimate from data.

The first is **True** if the word “pill” occurs in the e-mail and False if it doesn’t, and the second is **True** if the word “meeting” occurs and False if it doesn’t.

“pill”	“meeting”	label
T	F	Spam
T	F	Spam
F	T	Spam
T	F	Spam
F	F	Ham
F	F	Ham
F	T	Ham
T	T	Ham

question 14: We will build a naive Bayes classifier for this data. What is the defining property of naive Bayes? Why is it called “naive”?

The naive Bayes classifier assumes that the features are independent, *conditional on the class*. It is called naive, because this assumption is usually untrue. The resulting classifier nevertheless works well in many cases.

question 15: We build a naive Bayes classifier on this data, as described in the lecture. We get an email that contains both words. Which class does the classifier assign?

Call the class Y and the features X_p and X_m . The class probabilities are

$$p(Y | X_p, X_m) = \frac{p(X_p, X_m | Y)p(Y)}{p(X_p, X_m)} .$$

The class choice is

$$\arg \max_Y p(Y | X_p, X_m) = \arg \max_Y p(X_p, X_m | Y)p(Y) .$$

The *naive Bayes assumption* allows us to break the the probabilities up.

$$\arg \max_Y p(Y | X_p, X_m) = \arg \max_Y p(X_p | Y)p(X_m | Y)p(Y) .$$

We estimate the the class prior $P(Y)$ from the data (i.e. 0.5 for each class). We estimate the likelihood of the data given the class based on the relative frequencies in the data (i.e. $p(X_p | \text{Spam}) = \frac{3}{4}$). Since we only care about the class with the maximal probability, we can use Bayes rule without computing the denominator.

This gives us:

$$\begin{aligned} p(\text{Spam} \mid X_p = 1, X_m = 1) &\propto p(X_p = 1 \mid \text{Spam}) p(X_m = 1 \mid \text{Spam}) p(\text{Spam}) \\ &= \frac{3}{4} \frac{1}{4} \frac{1}{2} = \frac{3}{32} \end{aligned}$$

$$\begin{aligned} p(\text{Ham} \mid X_p = 1, X_m = 1) &\propto p(X_p = 1 \mid \text{Ham}) p(X_m = 1 \mid \text{Ham}) p(\text{Ham}) \\ &= \frac{1}{4} \frac{2}{4} \frac{1}{2} = \frac{2}{32} \end{aligned}$$

Spam gives a higher value (these are not probabilities, just values proportional to the true probabilities), so the classifier classifies the email as **Spam**.

question 16: Which probabilities does the classifier assign to each class? For full class probabilities, we need to apply Bayes' theorem including its denominator:

$$p(Y \mid X_p, X_m) = \frac{p(X_p, X_m \mid Y)p(Y)}{p(X_p, X_m)} .$$

The denominator expands as

$$p(X_p, X_m) = \sum_{Y \in \{\text{Spam}, \text{Ham}\}} p(X_p, X_m \mid Y)p(Y)$$

These are the two unnormalized probabilities ($\frac{3}{32}$ and $\frac{2}{32}$) that we've calculated already. Computing the proper probabilities is equivalent to normalizing the unnormalized ones. Thus, the class probabilities assigned are $\frac{3}{5}$ for **Spam** and $\frac{2}{5}$ for **Ham**.

To improve our accuracy, we add another feature:

"pill"	"meeting"	"hello"	label
T	F	F	Spam
T	F	F	Spam
F	T	F	Spam
T	F	F	Spam
F	F	F	Ham
F	F	F	Ham
F	T	F	Ham
T	T	T	Ham

question 17: For the class Spam, there are no emails recorded that contain the word "hello". Why is this a problem?

This makes our estimate of the probability $p(X_h = 1 \mid \text{Spam})$ zero. This causes the entire class probability to collapse to 0 even if the other features make Spam very likely. What solution is suggested in the slides?

To add *pseudo-observations*, so that for each feature each value is observed at least once for each class.

question 18: Implement this solution, and give the class probabilities for an email containing all three words.

After adding the pseudo observations, our dataset looks like this:

“pill”	“meeting”	“hello”	label
T	F	F	Spam
T	F	F	Spam
F	T	F	Spam
T	F	F	Spam
F	F	F	Ham
F	F	F	Ham
F	T	F	Ham
T	T	T	Ham
T	T	T	Spam
F	F	F	Spam
T	T	T	Ham
F	F	F	Ham

Note that we don’t add every possible combination of features (that would be 8 extra instances per class). We just make sure that for every feature there is one email that has F for that feature and Spam as a class, one email that has T for that feature and Spam as a class and the same for Ham.

This gives us:

$$\begin{aligned}
 & p(\text{Spam} \mid X_p = 1, X_m = 1, X_h = 1) \\
 & \propto p(X_p = 1 \mid \text{Spam}) p(X_m = 1 \mid \text{Spam}) p(X_h = 1 \mid \text{Spam}) p(\text{Spam}) \\
 & = \frac{4}{6} \frac{2}{6} \frac{1}{6} \frac{1}{2} = \frac{8}{6^3 2} \\
 & p(\text{Ham} \mid X_p = 1, X_m = 1, X_h = 1) \\
 & \propto p(X_p = 1 \mid \text{Ham}) p(X_m = 1 \mid \text{Ham}) p(X_h = 1 \mid \text{Ham}) p(\text{Ham}) \\
 & = \frac{2}{6} \frac{3}{6} \frac{2}{6} \frac{1}{2} = \frac{12}{6^3 2}
 \end{aligned}$$

Normalizing these gives us $\frac{2}{5}$ for Spam and $\frac{3}{5}$ for Ham.

5 Entropy

We define two probability distributions p and q on a set of four outcomes $\{a, b, c, d\}$.

$$\begin{array}{cccc} p(a) & p(b) & p(c) & p(d) \\ \hline 1/4 & 1/4 & 1/4 & 1/4 \\ \\ q(a) & q(b) & q(c) & q(d) \\ \hline 1/2 & 1/4 & 1/8 & 1/8 \end{array}$$

question 19: Could you simulate sampling from these distributions using coinflips as described in the lecture?

Yes. p can be simulated by assigning each unique sequence of two coinflips to one of the outcomes. For q , we can assign the following sequences to each outcome:

$$\begin{array}{cccc} q(a) & q(b) & q(c) & q(d) \\ \hline H & TH & TTH & TTT \end{array}$$

question 20: Compute the entropy of p and q .

$$\begin{aligned} H(p) &= - \sum_{y \in \{a,b,c,d\}} p(y) \log_2 p(y) \\ &= -4 \left(\frac{1}{4} \log \frac{1}{4} \right) = -\log 4 \times -1 = \log 4 = 2 \end{aligned}$$

$$\begin{aligned} H(q) &= - \sum_{y \in \{a,b,c,d\}} q(y) \log_2 q(y) \\ &= -\frac{1}{2} \log \frac{1}{2} - \frac{1}{4} \log \frac{1}{4} - \frac{1}{8} \log \frac{1}{8} - \frac{1}{8} \log \frac{1}{8} \\ &= \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{3}{8} = \frac{4}{8} + \frac{4}{8} + \frac{3}{8} + \frac{3}{8} = 1.75 \end{aligned}$$

The entropy of q is lower than the entropy of p . What does this tell you about the difference between the two distributions?

This means that p is more *uniform* than q . In other words, we have more *information* about what the outcome of a sample from q will be than we do about the outcome of a sample from p .

question 21: In the definition of entropy that we use (information entropy), the logarithms have base 2 (i.e. $\log_2$ instead of $\log_{10}$ or $\ln$). This follows directly from our decision to model probability distributions with coinflips. How?

The logarithm is defined as the expected code length under an idealized optimal binary code. If a binary code assigns a codeword of length of L to an outcome, the probability of sampling that outcome by generating a random code by flipping a coin is $(\frac{1}{2})^L = 2^{-L}$. If we reverse this, to find the optimal code belonging to an event with probability $p(x)$, we get $L = -\log_2 p(x)$